UP1: FICHEROS

TAREA 3 – CATÁLOGO EN MEMORIA Y PRIMERAS PRUEBAS CON JUNIT

En la tarea anterior creaste la clase Videojuego y una clase Main desde la que construiste y mostraste dos videojuegos.

Ahora vas a continuar trabajando sobre ese mismo proyecto. Crearás un catálogo en el que podrás guardar varios videojuegos y modificarás Main para utilizar la aplicación mediante un pequeño menú. También prepararás tus primeras pruebas automáticas con JUnit.

Recuerda que, igual que en la tarea anterior, deberás escribir el código sin utilizar Lombok, asistentes de inteligencia artificial ni sugerencias automáticas de bloques de código. La idea es que practiques y comprendas cada parte del programa que estás construyendo.

Objetivos

- Utilizar una colección para almacenar objetos.
- Añadir, buscar y eliminar videojuegos.
- Crear un menú de opciones en la consola.
- Organizar el programa para que cada clase tenga una función concreta.
- Crear y ejecutar pruebas automáticas con JUnit.
- Comprobar qué sucede con los datos cuando finaliza el programa.
- Guardar los cambios del proyecto mediante commits de Git.

1 Crear la clase Catalogo

Dentro del paquete `es.iesmutxamel.ludoteca`, crea una clase llamada `Catalogo`.

Esta clase se encargará de almacenar y gestionar los videojuegos de la ludoteca. Para ello, utilizará un `ArrayList<Videojuego>` como atributo privado, que deberás inicializar en el constructor.

La clase deberá permitir:

- Añadir un videojuego, evitando que se repitan los identificadores.
- Buscar un videojuego por su identificador.
- Eliminar un videojuego.
- Consultar cuántos videojuegos hay almacenados.
- Obtener los videojuegos guardados en el catálogo.

Utiliza nombres claros para los métodos, como agregarVideojuego, buscarPorId, eliminarPorId y obtenerCantidad, y decide qué parámetros y tipos de retorno necesita cada uno.

La clase Catalogo se ocupará únicamente de gestionar los videojuegos. No deberá mostrar el menú ni solicitar datos al usuario; esa parte se realizará desde Main.

2 Modificar la clase Main

Modifica la clase Main para que sea el punto de entrada de la aplicación y se encargue de la comunicación con el usuario.

Al comenzar el programa, deberás crear un catálogo y añadir los dos videojuegos que utilizaste en la tarea anterior.

A continuación, muestra un menú que permita realizar las siguientes operaciones:

LUDOTECA IES

1. Mostrar todos los videojuegos

2. Añadir un videojuego

3. Mostrar un resumen del catálogo

4. Consultar videojuegos con pocas unidades

0. Salir

>> Selecciona una opción:

El programa deberá volver a mostrar el menú después de cada operación y continuar ejecutándose hasta que se seleccione la opción de salir.

Opción 1: Mostrar todos los videojuegos

Muestra todos los videojuegos almacenados en el catálogo. Si no hay ninguno, informa de que el catálogo está vacío.

Opción 2: Añadir un videojuego

Solicita los datos necesarios para crear un videojuego y añádelo al catálogo.

No se podrá añadir un videojuego si ya existe otro con el mismo identificador.

Opción 3: Mostrar un resumen del catálogo

Muestra la siguiente información:

- Número de videojuegos diferentes.
- Número total de unidades disponibles.

- Valor total del stock, calculado a partir del precio y las unidades de cada videojuego.

Opción 4: Consultar videojuegos con pocas unidades

Solicita al usuario la cantidad máxima de unidades que quiere consultar.

Por ejemplo, si introduce 3, el programa deberá mostrar todos los videojuegos que tengan un stock de 0, 1, 2 o 3 unidades. De esta manera, se pueden identificar los videojuegos que necesitan reponerse.

Si ningún videojuego cumple la condición, muestra un mensaje indicándolo.

Opción 0: Salir

Finaliza la aplicación mostrando un mensaje de despedida.

Utiliza Scanner para recoger los datos introducidos por el usuario. Puedes elegir el tipo de bucle y la estructura de selección que consideres más adecuados.

Las operaciones relacionadas con los videojuegos deberán realizarse mediante la clase Catalogo. Main se encargará de solicitar los datos, mostrar el menú y presentar los resultados.

Validación de los datos

Aplica los conocimientos adquiridos en primero para controlar las entradas del usuario. La aplicación no debe finalizar de forma inesperada cuando se introduzca un dato incorrecto.

Como mínimo, controla estas situaciones:

- Se introduce texto cuando se espera un número.
- Se selecciona una opción que no existe.
- El identificador, el precio o el stock tienen un valor negativo.
- El título o la plataforma están vacíos.
- Se intenta añadir un identificador repetido.
- El límite de stock introducido es negativo.

Cuando se produzca un error, muestra un mensaje claro y permite que el usuario vuelva a intentarlo o regrese al menú.

3 Crear las primeras pruebas con Junit

Preparar JUnit 5

JUnit no se instala como un programa independiente. Como el proyecto utiliza Maven, debes añadirlo como una dependencia.

Abre el archivo pom.xml y añade este bloque dentro de <project>:

```
<dependencies>
    <dependency>
        <groupId>org.junit.jupiter</groupId>
        <artifactId>junit-jupiter</artifactId>
        <version>5.13.1</version>
        <scope>test</scope>
    </dependency>
</dependencies>
```

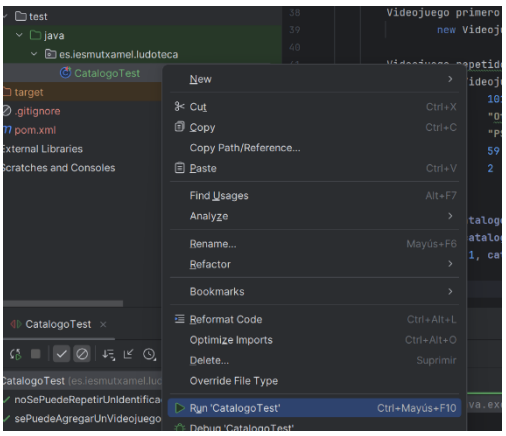
Si el archivo ya contiene un apartado <dependencies>, añade solamente el bloque <dependency> dentro de él.

Después:

1. Guarda el archivo pom.xml.
2. Pulsa Load Maven Changes cuando IntelliJ muestre el aviso.
3. Espera a que Maven descargue JUnit. La primera vez necesitarás conexión a Internet.
4. Crea la clase CatalogoTest dentro de:

   `src/test/java/es/iesmutxamel/ludoteca`

5. Comprueba que IntelliJ reconoce las anotaciones `@Test` y `@BeforeEach`.
6. Para ejecutar las pruebas, pulsa el triángulo verde situado junto al nombre de la clase CatalogoTest.

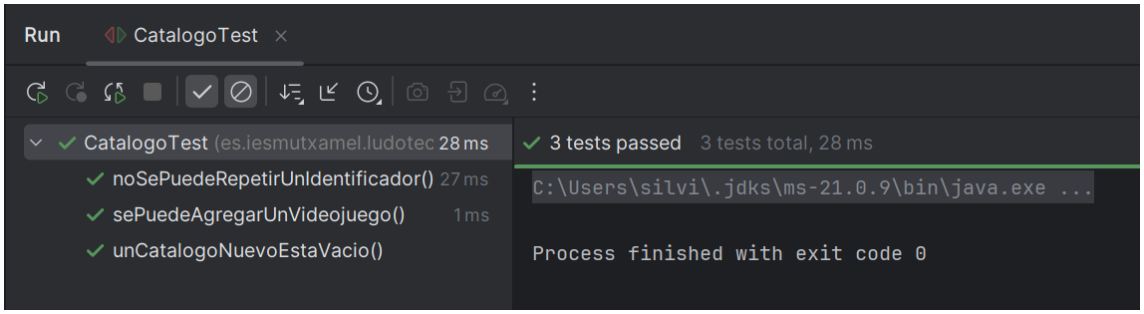


Manual oficial de JUnit en IntelliJ IDEA

Crear los test

- Comprobar que un catálogo recién creado no contiene videojuegos.
- Comprobar que se puede añadir un videojuego y que la cantidad almacenada pasa a ser uno.
- Comprobar que no se pueden añadir dos videojuegos con el mismo identificador.

Pon a cada test un nombre que permita entender qué estás comprobando y ejecútalos hasta que los tres aparezcan PASSED, en verde, en IntelliJ IDEA.



No necesitas crear pruebas para la clase Main, el menú ni las entradas realizadas mediante Scanner.

4 Guardar los avances con Git

Continúas trabajando con el repositorio Git local que creaste anteriormente. No debes volver a ejecutar git init ni crear todavía un repositorio en GitHub. Más adelante conectarás este repositorio local con GitHub y subirás todo el historial del proyecto.

Guarda los avances en momentos diferentes. No realices un único commit con todo el trabajo al final.

Como mínimo, deberás realizar estos tres commits:

- Crear la clase Catalogo
- Añadir el menú y las validaciones
- Añadir las pruebas de Catalogo

Antes de cada commit, utiliza: git status

Comprueba qué archivos has modificado y asegúrate de que no aparecen las carpetas .idea, target u out.

Después, añade los cambios y realiza el commit siguiendo el procedimiento utilizado en la tarea anterior.

Al terminar, ejecuta: git log --oneline

El historial deberá mostrar el primer commit de la Tarea 1 y, como mínimo, los tres nuevos commits de esta tarea.

Finalmente, vuelve a ejecutar: git status

Si todo se ha guardado correctamente, Git deberá indicar que no hay cambios pendientes. El resultado será similar a: nothing to commit, working tree clean

5 Entrega

Incluye en el documento de entrega:

- Una captura del menú funcionando.
- Una captura del resumen del catálogo o de la consulta de pocas unidades.
- Una captura en la que se vea cómo se controla una entrada incorrecta.
- Una captura de los tres tests de JUnit superados.
- Una captura de git log --oneline.
- Una captura final de git status sin cambios pendientes.

Las capturas deben ser legibles. Debajo de cada una, añade una frase breve explicando qué estás comprobando.